

Replacement methods implementation

April 18, 2023

Name	ID	Department
Abdelrhman Mohamed Abdelhady Hodib	2022513643	Intelligent system

```
[9]: # Note : we use frame size = 3 in all the algorithms
```

FIFO Algorithm

```
[1]: def fifo(page_references):  
  
    """  
    This function how work ?  
  
    it takes page references as an argument as list  
  
    every page passes through the loop  
  
    and checks if the page is in the frames or not  
  
    if it is in the frames then page faults is constant  
  
    else we update the page faults  
  
    if the page already not in the frames  
  
    we check if there is empty frame or not  
  
    yes , we will append that page  
  
    no, we will remove the first page in the list then append that page  
  
    After we end our loop we return the NO. of page faults and  
  
    shape of last frame  
  
    """  
    frames = []  
    page_faults = 0
```

```

for page in page_references:
    if page not in frames:
        page_faults += 1
        if len(frames) < 3:
            frames.append(page)
        else:
            frames.pop(0)
            frames.append(page)
    return page_faults, frames
# Example

page_references = [1, 2, 3, 4, 1, 2, 5, 6, 1, 2, 3, 7, 8, 7, 8, 9, 7, 8, 9, 5]

faults_frame = fifo(page_references)
print("Total Page Faults and the last frame will be:", faults_frame)

```

Total Page Faults and the last frame will be: (15, [8, 9, 5])

LFU Algorithm

```

[4]: def lfu(page_references):
    """
    This function how work ?

    it takes page references as an argument as list

    every page passes through the loop

    and checks if the page is in the frames or not

    if it is in the frames then page faults is constant and update the
    ↪ frequency of that page

    else we update the page faults

    if the page already not in the frames

    we check if there is empty frame or not

    yes , we will append that page

    and update frequency of that page to be 1 in the frequency dictionary

    no, we will remove the page with lowest frequency by

    checking the key (page) with the lowest frequency in the dictionary
    """

```

then we remove it from the frames list and from the key from the frequency_ dict.

then add the page and make their frequency value is 1

After we end our loop we return the NO. of page faults and

shape of last frame

```
"""
frames = []
page_faults = 0
frequency = {}

for page in page_references:
    if page not in frames:
        page_faults += 1
        if len(frames) < 3:
            frames.append(page)
            frequency[page] = 1
        else:
            # Frequency lowest
            min_freq_page = min(frames, key=frequency.get)
            frames.remove(min_freq_page)
            del frequency[min_freq_page]
            frames.append(page)
            frequency[page] = 1
    else:
        frequency[page] += 1

return page_faults, frames

# Example
page_references = [1, 2, 3, 4, 1, 2, 5, 6, 1, 2, 3, 7, 8, 7, 8, 9, 7, 8, 9, 5]
faults_frame = lfu(page_references)
print("Total Page Faults and the last frame will be:", faults_frame)
```

Total Page Faults and the last frame will be: (15, [7, 8, 5])

MFU Algorithm

```
[6]: def mfu(page_references):
    """
    This function how work ?

    Same as LFU we only remove the page with the highest frequency
```

```

"""
frames = []
page_faults = 0
frequency = {}

for page in page_references:
    if page not in frames:
        page_faults += 1
        if len(frames) < 3:
            frames.append(page)
            frequency[page] = 1
        else:
            # Frequency highest
            max_freq_page = max(frames, key=frequency.get)
            frames.remove(max_freq_page)
            del frequency[max_freq_page]
            frames.append(page)
            frequency[page] = 1
    else:
        frequency[page] += 1

return page_faults, frames

# Example
page_references = [1, 2, 3, 4, 1, 2, 5, 6, 1, 2, 3, 7, 8, 7, 8, 9, 7, 8, 9, 5]
faults_frame = mfu(page_references)
print("Total Page Faults and the last frame will be:", faults_frame)

```

Total Page Faults and the last frame will be: (17, [7, 8, 5])

Clock Algorithm

```

[8]: def clock(page_references):
    """
        frames is a list that represents the frames, with -1 indicating an empty
        ↪ frame

        reference_bit is a list that keeps track of the reference bit for each
        ↪ frame, initialized to 0

        when a page fault occurs the algorithm scans through the frames using the
        ↪ pointer and checks the reference bit of each frame.

        if the reference bit is 0, the page is replaced in that frame and the
        ↪ reference bit is set to 1
    """

```

*if the reference bit is 1, it is reset to 0, and the pointer moves to the
→next frame in a circular manner*

*if the requested page is already in a frame, its reference bit is set to 1
→to indicate it was recently referenced.*

```
"""
frames = [-1] * 3
page_faults = 0
pointer = 0
reference_bit = [0] * 3

for page in page_references:
    if page not in frames:
        page_faults += 1
        while True:
            if reference_bit[pointer] == 0:
                frames[pointer] = page
                reference_bit[pointer] = 1
                pointer = (pointer + 1) % 3
                break
            else:
                reference_bit[pointer] = 0
                pointer = (pointer + 1) % 3
        else:
            reference_bit[frames.index(page)] = 1
    return page_faults, frames
# Example
page_references = [1, 2, 3, 4, 1, 2, 5, 6, 1, 2, 3, 7, 8, 7, 8, 9, 7, 8, 9, 5]
faults_frame = clock(page_references)
print("Total Page Faults and the last frame will be:", faults_frame)
```

Total Page Faults and the last frame will be: (15, [8, 9, 5])